Final Report: TCP Congestion and Network Performance Analysis

CMPE 206 · Oct 24, 2026 · @Lucas

Abstract. At a shared bottleneck, the queue mattered more than the senders' congestion control algorithm (CCA). In 492 automated trials on a Mininet dumbbell with a 50 Mbps router bottleneck, Reno, CUBIC and BBR competed with each other and with UDP under pfifo, RED and fq_codel. On a plain FIFO, BBR took 89–99% of the link from CUBIC, line-rate UDP left TCP under 0.1 Mbps, and CUBIC in a deep buffer added up to 494 ms of delay. fq_codel kept delay near zero and Jain's index at or above 0.97.

1. Introduction and problem statement

Three things split a shared bottleneck between users: each sender's CCA, the bottleneck queue, and each path's round-trip time (RTT). Reno and CUBIC fill buffers until packets drop, the root of bufferbloat [14]. BBR paces at a bandwidth estimate and can crowd them out, and UDP never backs off. **Research question:** which CCA and queue should an operator deploy as flow count, RTT, buffer size and UDP load change, judged by throughput, fairness and latency?

2. Background

Reno [1, 2] adds one segment per RTT to its congestion window (cwnd) and halves it on loss. *CUBIC* [3], the Linux default, grows cwnd as a cubic function of time since the last loss, independent of RTT, and cuts it to 0.7×. *BBR* [4] (v1) ignores loss: it paces at its bottleneck-bandwidth estimate, keeps in-flight data near 2 × bandwidth × minimum RTT, and every 10 s drains the queue for about 200 ms (ProbeRTT). *pfifo* (DropTail) drops arrivals at a fixed limit, and *RED* [10] drops early and at random as the average queue grows. *fq_codel* [8] serves per-flow queues round-robin, with CoDel [5] dropping once packets have waited over 5 ms for 100 ms.

At 50 Mbps and 40 ms the bandwidth-delay product (BDP = R × RTT) is 250 KB, or 167 packets. One BDP of buffer keeps the link busy after a single loss-based flow halves; many flows need less [7]. A full buffer of B bytes adds B / R to every RTT. Jain's index [6], $J = (\Sigma x_i)^2 / (n \cdot \Sigma x_i^2)$, measures fairness: 1 for equal shares, 1/n when one flow takes everything.

3. Network architecture and design

Senders h1–h4 sit on switch s1 and receivers h5–h8 on switch s2, with router r1 between them (Figure 1). Only r1's port r1-eth1 is shaped. A separate probe pair, hp → hr, measures RTT while the link is loaded.

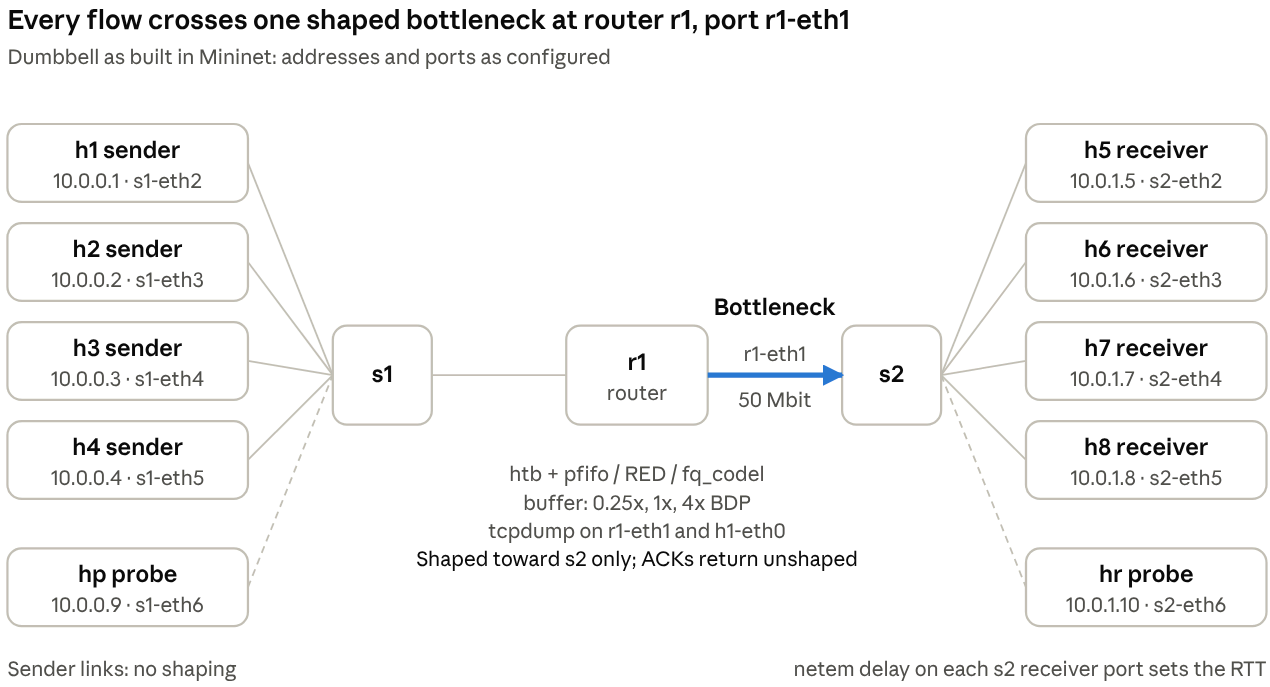


Figure 1 · dumbbell topology: 4 senders, 4 receivers, a probe pair, 2 switches, router r1 as the bottleneck

On r1-eth1, htb at 50 Mbit feeds one child queue (pfifo, RED or fq_codel). It is the only queue on the path, so the buffer is exactly as configured. netem delay on each receiver's s2 port sets per-path RTTs, and ACKs return unshaped. My first design shaped a switch port instead and failed validation: 60 Mbps of UDP arrived as 48.7 Mbps with no loss. Across veth pairs and Open vSwitch a packet stays charged to its socket, so the full queue throttled the sender instead of dropping. Routing through r1 releases the packet, and the same test then lost 19%.

4. Implementation methodology

Platform. A 4-vCPU, 4 GB UTM virtual machine on an Apple M2 ran Ubuntu 26.04.1 (kernel 7.0), Mininet 2.3.0 [9], Open vSwitch 3.7.1, iperf 3.20 [16] and Wireshark 4.6.4; analysis used Python and pandas. Buffers hold k × BDP / 1500 B packets (11 to 2,500), and RED's thresholds sit at 1/6 and 1/2 of the limit. Offloads are off, so the queue sees real 1514-byte frames.

Automation and metrics. `run.py` expands each scenario's YAML grid, shuffles the configurations and interleaves their trials. Every trial builds a fresh network and checks the idle RTT. iperf3 then runs for 60 s after a 5 s omit period that skips slow start, while `ping` samples every 0.2 s, `tc -s qdisc` every 0.5 s and `ss` every 0.1 s; 16 configurations were also captured in Wireshark. Metrics cover only the time all flows run (5–65 s, or 5–60 s in S2, where UDP stops first). Utilization is bytes sent on r1-eth1 ÷ (R × time). Queueing delay is the loaded minus the idle median probe RTT, and jitter is iperf3's RFC 3550 estimator [15] averaged per second. Values are 5-trial means (8 trials for four noisy configurations) with 95% intervals, mostly under 1% of the mean.

Validation and limits. Nine checks passed before any experiment; for example, a 40 ms path measured a 40.2 ms minimum RTT, and the queue held exactly its limit. Since htb counts headers, TCP goodput tops out at 50 × 1448 / 1514 = 47.8 Mbps. VM timing leaves about ±4 ms of noise in queueing delay, and VM batching makes bursts larger.

5. Experimental scenarios

All three scenarios run Reno, CUBIC and BBR over the same 50 Mbps bottleneck (Table 1).

Scenario	Grid (defaults: 40 ms RTT, 1× BDP, pfifo)	Configs	Trials
S1 Competing TCP flows	1 flow per CCA; 2 or 4 flows × equal or mixed RTTs (20–80 ms) × Reno, CUBIC, BBR or CUBIC + BBR × pfifo, fq_codel	35	178
S2 TCP + UDP	3 TCP + 1 UDP flow at 25, 50, 75, 100% of R × CCA × pfifo, fq_codel	24	131
S3 Buffer and delay	1 flow: 0.25×, 1×, 4× BDP × RTT 10, 50, 150 ms × CCA; 2 CUBIC + 2 BBR at 50 ms: buffer × pfifo, RED, fq_codel	36	183

Table 1. 95 configurations and 492 trials, including runs where BBR (S1) or UDP (S2) starts late. No loss is injected; all drops happen at the queue.

6. Results

S1: competing flows. Any single flow filled the link (Reno and CUBIC 99.97%, BBR 97.5%), but CUBIC queued 27.9 ms, Reno 22.6 ms and BBR only 0.8 ms. Same-CCA flows with equal RTTs shared fairly (Jain ⩾ 0.95). Mixed CCAs on pfifo did not: CUBIC kept 10.6%

against one BBR flow, 1.9–3.3% in a 2 + 2 mix, and about 5 Mbps when BBR joined late. Under mixed RTTs (Figure 2), Reno gave its 20 ms flow 37.6% (Jain 0.88) and CUBIC was fairer at 0.95, while BBR's 80 ms flow took 39% against 7% (0.74).

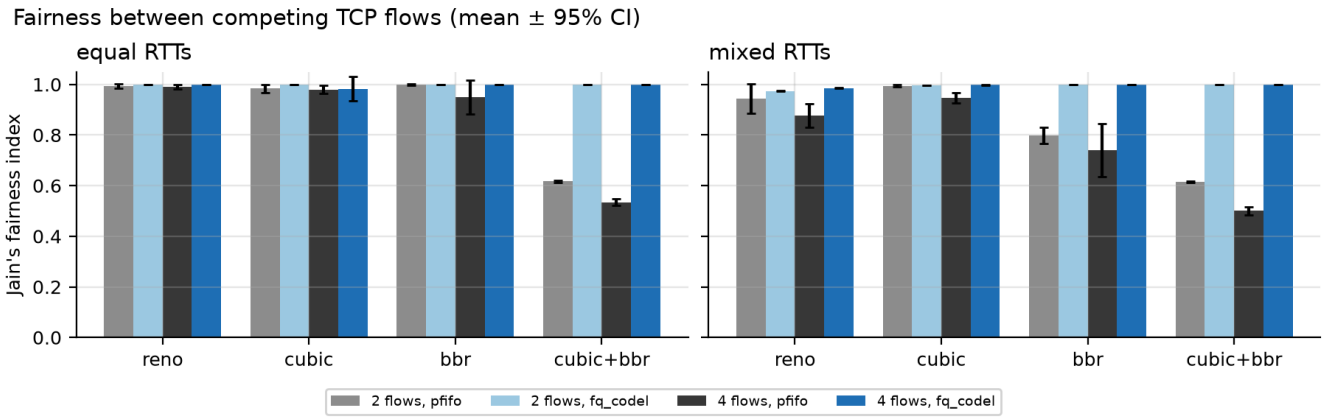


Figure 2. Jain's index in S1. CUBIC + BBR on pfifo is the outlier; fq_codel stays at 0.97–1.00.

S2: TCP + UDP. On pfifo, three CUBIC flows together got 35.5, 23.1, 10.7 and 0.05 Mbps as UDP rose from 25% to 100% of R, and Reno and BBR were within 0.9 Mbps of that. fq_codel held TCP at 33–35 Mbps by dropping UDP's excess, so UDP loss reached 75%. In exchange, UDP jitter at 75–100% load rose from at most 0.11 ms to 0.6–0.8 ms (Figure 3).

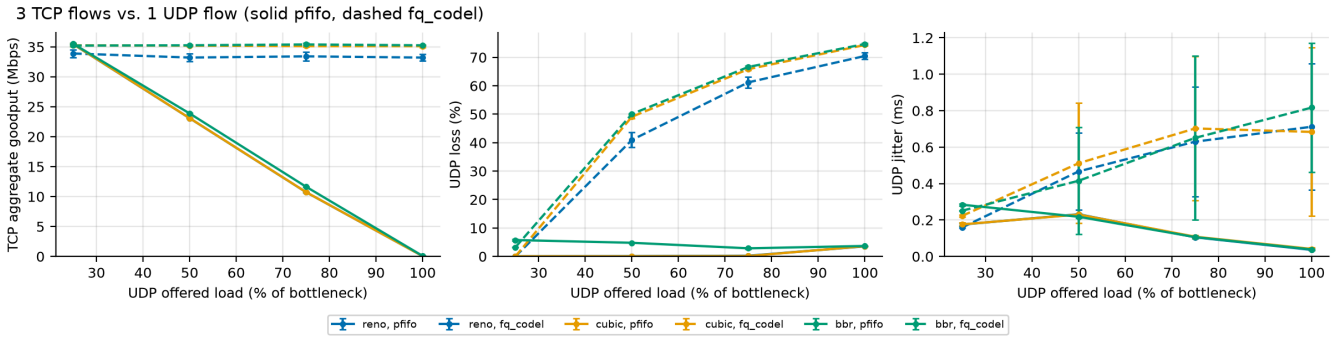


Figure 3. TCP goodput, UDP loss and UDP jitter versus UDP load. fq_codel keeps TCP at about three quarters of the link.

S3: buffers and RTT. At 1× BDP or more, every CCA stayed above 96% utilization. At 0.25× BDP Reno dropped to 82–87%, while CUBIC kept 98% and BBR 95–97%; only the 11-packet buffer at 10 ms broke this pattern, with Reno and CUBIC both at 23% (Figure 4). On pfifo, CUBIC's share against BBR went from 1.4% to 1.8% to 58.9% as the buffer grew from 0.25× to 4× BDP. RED left it about 2% and fq_codel about 48% (Figure 5). At 4× BDP, CUBIC added 34.5, 162 and 494 ms of delay at RTTs of 10, 50 and 150 ms, Reno 28, 127 and 245 ms, and BBR under 3 ms. Four mixed flows saw 164 ms on pfifo, 48 ms on RED and about 0 on fq_codel.

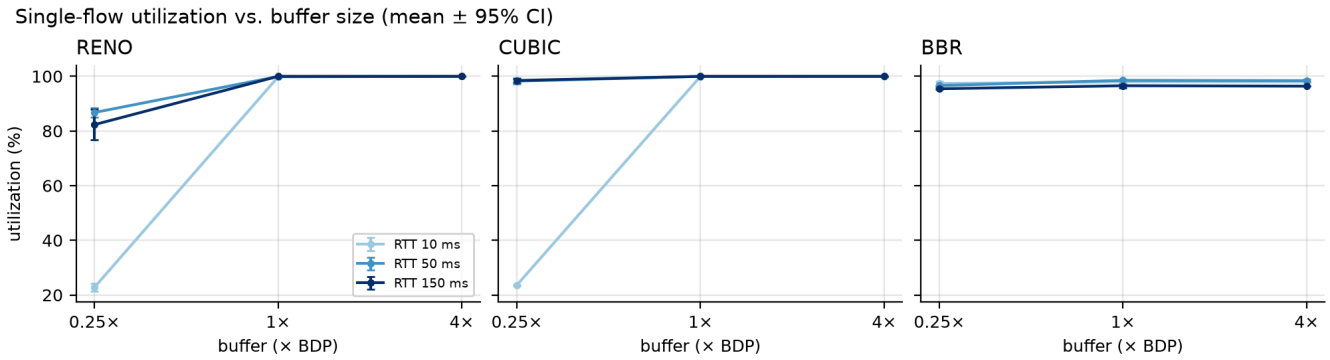


Figure 4. Single-flow utilization by buffer size. Only shallow buffers cost throughput, and only for the loss-based CCAs.

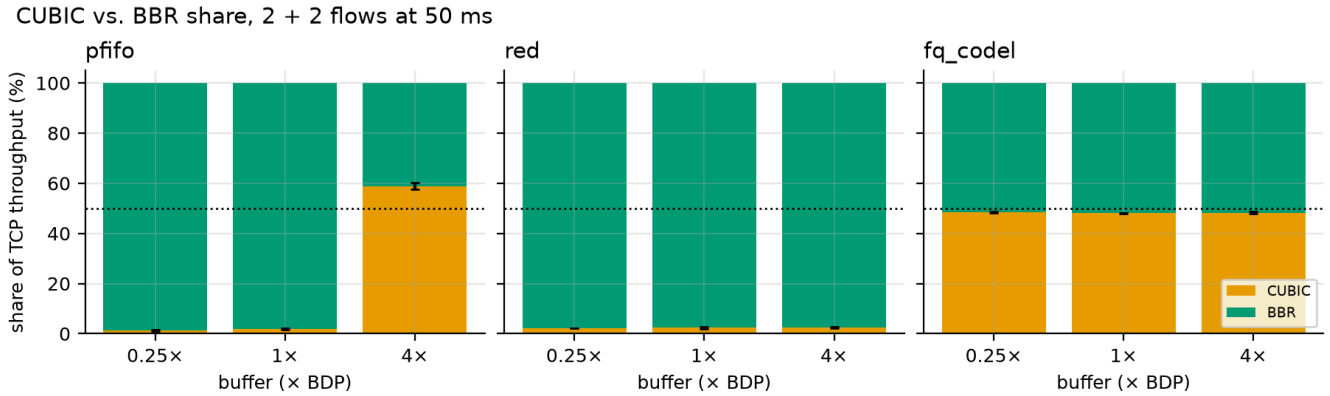


Figure 5. CUBIC's share against BBR. On pfifo it follows buffer depth; fq_codel evens it out.

7. Analysis and discussion

Two properties explain most results: loss response and flow isolation (Table 2).

Hypothesis (from the proposal)	Verdict
H1: ⩾ 90% utilization at ⩾ 1× BDP; Reno 85–90% at 0.25×	Partly: plus an 11-packet collapse
H2: Reno favors short RTTs more than CUBIC	Supported; BBR reverses it
H3: BBR wins shallow buffers, CUBIC deep ones	Supported on pfifo, not RED
H4: line-rate UDP starves TCP; fq_codel fixes it	Supported
H5: a deep FIFO adds about its drain time	CUBIC yes; Reno partly

Table 2. Proposal hypotheses and what the data showed.

H1 (utilization). The link idles whenever cwnd falls below one BDP. With b BDPs of buffer, cwnd peaks at (1 + b) BDP. Reno halves and regrows linearly, which for b = 0.25 predicts 89% utilization; I measured 86.7%, and the capture shows bytes in flight dipping to 186 KB against a 312 KB BDP. CUBIC's 0.7× cut predicts 98% (measured 98.2%). At 150 ms Reno needs about 40 s to regrow, so a trial holds one or two cycles. BBR ignores loss; ProbeRTT explains its 2–3% shortfall. The 11-packet collapse was not predicted. Unpaced bursts overflow the queue (about 1,000 drops per trial) and pin cwnd near 11 packets against a 42-packet BDP, while paced BBR lost 9,500 packets and still kept 97%. Tiny buffers need smooth traffic [11], and VM batching probably exaggerates the effect.

H2 and H3 (sharing between flows). After a shared loss, Reno's 20 ms flow regrows about twice as fast as its 80 ms flow (effective RTTs of 49 and 109 ms). CUBIC's time-based growth narrows the gap. BBR caps in-flight data by its own minimum RTT, so longer-RTT flows hold more of the FIFO [12, 13]. In shallow buffers BBR sends through the losses it causes while CUBIC backs off. At 4× BDP the roles flip: CUBIC fills the queue before any loss, BBR's cap does not grow with queueing delay, and CUBIC wins. RED drops from both, but only CUBIC responds. Under fq_codel's round-robin, window size no longer matters.

H4 (UDP). In a shared queue, every drop halves a TCP flow but UDP keeps its rate, so TCP gets exactly what UDP leaves. With 1242-byte UDP frames that predicts 35.4, 23.1, 10.7 and 0 Mbps, within 0.1 Mbps of the measurements, and at 100% load UDP's 3.4% excess over the link matches its 3.6% loss. In the late-UDP capture TCP collapsed within 0.3 s, and 21% of its remaining packets were retransmissions (Figure 6). fq_codel gives each flow a quarter of the link; interleaving UDP with TCP bursts raises its jitter. BBR fares no better: at 25% load it caused 5.8% UDP loss (CUBIC 0.06%) by filling the FIFO.

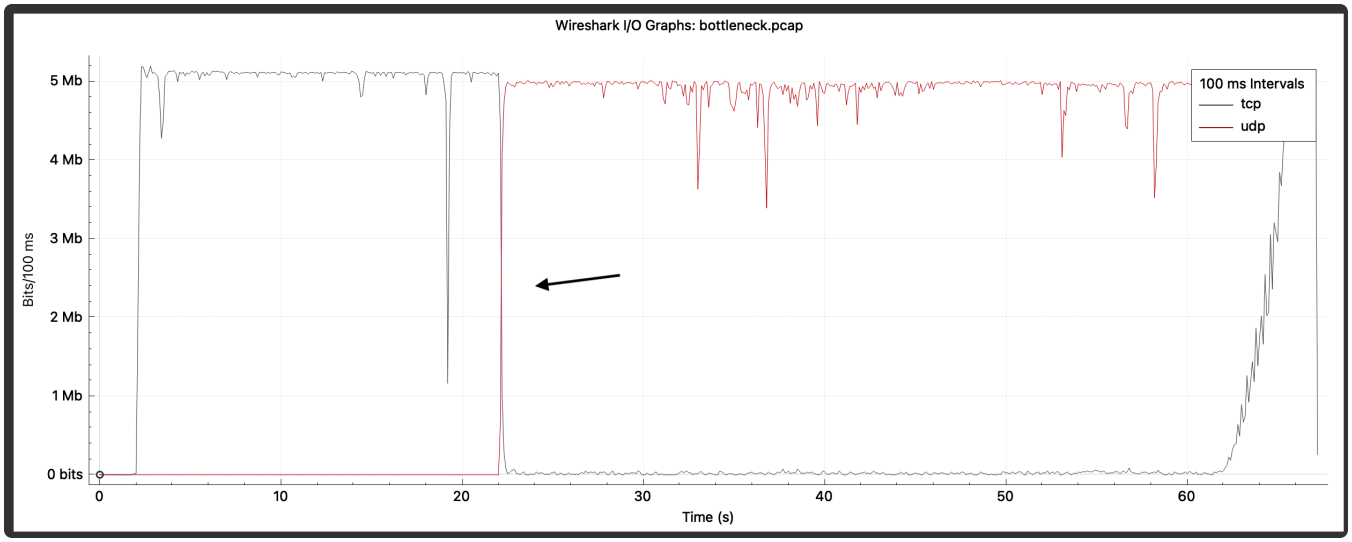


Figure 6. Wireshark I/O graph, udp100-cubic-pfifo with late UDP (trial 1). TCP collapses when UDP joins at 22 s (capture time) and recovers only after UDP stops.

H5 (queueing delay). A 4× BDP buffer drains in 4 × RTT: 40, 200 and 600 ms here. CUBIC's 0.7× cut keeps it 80–86% full (captured RTT 175–250 ms on a 50 ms path). Reno cycling between 2.5 and 5 BDP should keep it 69% full, which matches the 64–71% at 10 and 50 ms; at 150 ms it never regrows within 60 s and reaches only 41%. BBR holds about one BDP in flight and CoDel drops after 5 ms, so neither builds a standing queue.

Testbed lessons. Besides the router fix (Section 3), two measurement choices mattered: utilization must be counted on the wire, and metrics must cover only the time all flows overlap. My first S2 analysis did not, and TCP beside line-rate UDP appeared to get 2.2 Mbps that actually arrived after UDP stopped.

8. Design recommendations

Changing the bottleneck queue helps more than changing any CCA. The CCA choice matters only where a FIFO must stay (Table 3).

Condition	Recommendation	Evidence
Operator controls the bottleneck	**fq_codel**, any CCA	Delay ~0 ms; Jain ⩾ 0.97
Unresponsive UDP	**fq_codel**; no CCA helps	TCP < 0.1 Mbps on pfifo
Mixed CCAs or RTTs on a FIFO	**fq_codel**, not RED	CUBIC 1.4–10.6% vs BBR
Deep FIFO (⩾ 4× BDP)	**fq_codel**, ~1× BDP, or BBR	CUBIC +494 ms; BBR < 3 ms
~1× BDP FIFO, one CCA	**BBR** for latency, **CUBIC** for throughput	BBR 0–3 ms; CUBIC ~100%
Shallow FIFO (⩽ 0.25× BDP)	**BBR**, or CUBIC at RTT ⩾ 50 ms	Reno 82–87%; 23% at 10 ms

Table 3. Recommendations for one 50 Mbps bottleneck in a VM, with BBRv1 and 60 s flows. Under fq_codel, Reno loses about 5%.

9. Conclusion

Across 492 trials, the bottleneck queue shaped fairness and delay more than the sender's CCA. On a FIFO, BBR and UDP take bandwidth from flows that back off on loss, and loss-based senders turn deep buffers into hundreds of milliseconds of delay. fq_codel removed all of these effects at a cost of under 1 ms of UDP jitter. If a FIFO must stay, BBR suits latency and tiny buffers when every sender uses it, and CUBIC suits long or mixed RTTs. Future work: a two-bottleneck parking-lot topology, BBRv3, and hardware without VM batching.

10. References

1. V. Jacobson, "Congestion Avoidance and Control," ACM SIGCOMM, 1988.
2. M. Allman, V. Paxson, E. Blanton, RFC 5681: TCP Congestion Control, 2009.
3. S. Ha, I. Rhee, L. Xu, "CUBIC: A New TCP-Friendly High-Speed TCP Variant," ACM SIGOPS OSR, 2008.
4. N. Cardwell et al., "BBR: Congestion-Based Congestion Control," ACM Queue, 2016.
5. K. Nichols, V. Jacobson, "Controlling Queue Delay," ACM Queue, 2012.
6. R. Jain, D. Chiu, W. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," DEC TR-301, 1984.
7. G. Appenzeller, I. Keslassy, N. McKeown, "Sizing Router Buffers," ACM SIGCOMM, 2004.
8. T. Høiland-Jørgensen et al., RFC 8290: The Flow Queue CoDel Packet Scheduler, 2018.
9. N. Handigol et al., "Reproducible Network Experiments Using Container-Based Emulation," ACM CoNEXT, 2012.
10. S. Floyd, V. Jacobson, "Random Early Detection Gateways for Congestion Avoidance," IEEE/ACM Trans. Networking, 1993.
11. M. Enachescu et al., "Routers with Very Small Buffers," IEEE INFOCOM, 2006.
12. M. Hock, R. Bless, M. Zitterbart, "Experimental Evaluation of BBR Congestion Control," IEEE ICNP, 2017.
13. R. Ware et al., "Modeling BBR's Interactions with Loss-Based Congestion Control," ACM IMC, 2019.
14. J. Gettys, K. Nichols, "Bufferbloat: Dark Buffers in the Internet," ACM Queue, 2011.
15. H. Schulzrinne et al., RFC 3550: RTP, 2003.
16. ESnet, iperf3.